Introduction to Debugging

Debugging on ACEnet Systems

Michelle Shaw

Computational Research Consultant, ACEnet-MUN

February 20, 2009

Outline

- Introduction
- The Debug Process...
- Tools at ACEnet
- Examples
- Further Reading

Introduction

- debugging - the process of finding and fixing defects in your code
- defects lead to errors in program execution
- errors cause the state of the program to be altered from it's intended behavior
- these errors may eventually lead to program failure
- classes of *defects* - syntax, logical, algorithm/design
- syntax errors usually detected by compiler

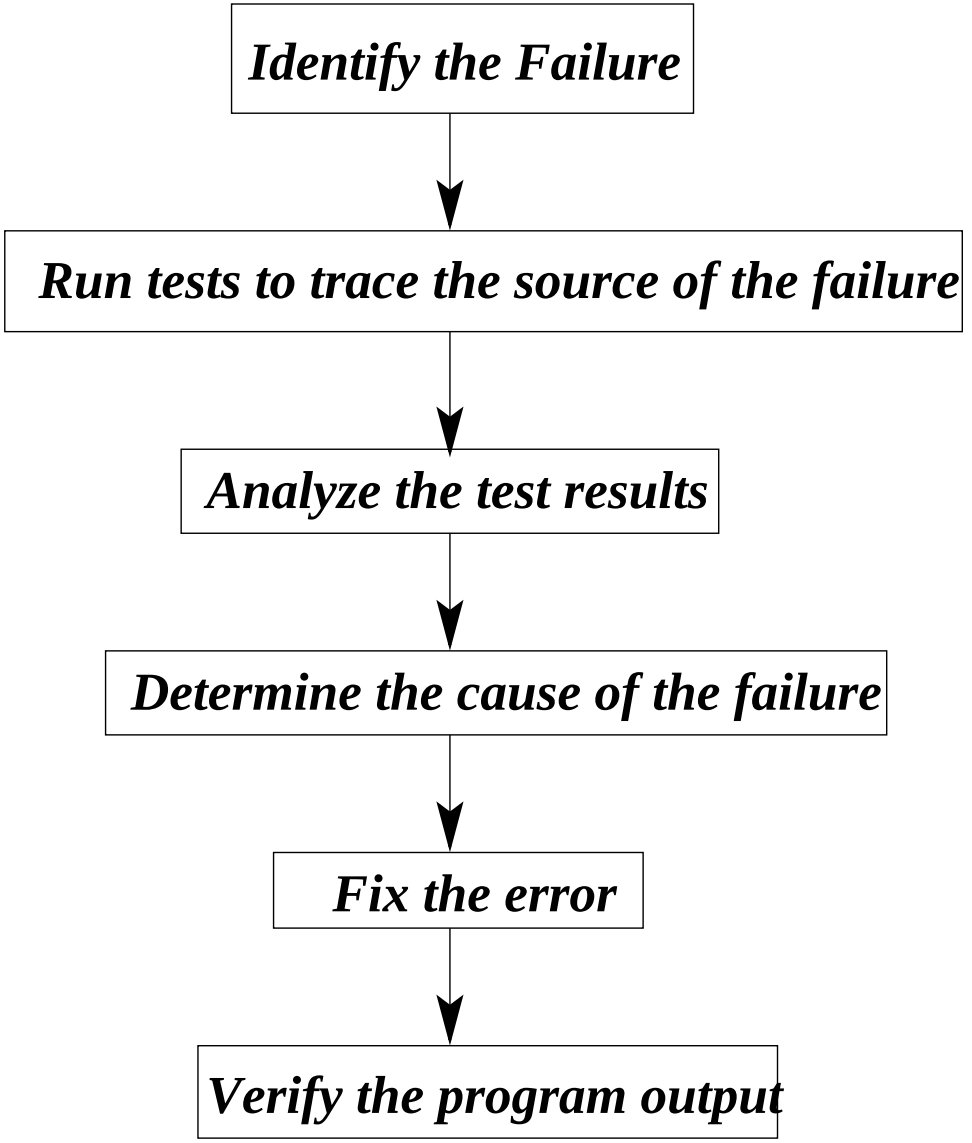
Introduction

- logical errors detected at runtime
- usually result in incorrect program output
- algorithm/design errors the most difficult to find
  - due to flaws in the design of the program
- the process of debugging involves determining what went wrong and where

Introduction

- first one must isolate what part of the program is related to the defect/failure
- the chain of program events that leads to the change in program state must also be determined
- these two steps take up more than half of a programmer's work time (sometimes longer)
- effective debugging techniques can reduce this number greatly
- they require both an understanding of the code and how it interacts with its environment

The Process of Fixing Defects



Identifying the Failure

- Note program behavior -
  - does it crash? If so, is there any communication from the OS?
  - very odd output?
  - taking too long?
  - strange program behavior?
- understand program's intended behavior well!!!!
- compare to determine the failure

Data Collection...

- attempt to reproduce the conditions of the failure
- document conditions under which the failure occurs
- simplify the input and re-run - does the failure again occur?
- run other examples by varying one or two input parameters at a time
- document, document, document...
- *trace* through the code by examining different program states along execution path

Tracing Through the Program

- can be done by hand with print statements or with a *debugger*
- definition of the *state* using:
  - variable and array values
  - presence of exceptions (or *signals*)
  - *execution stack* contents
  - layout of memory and register values
- for parallel programs, these values can be output for each execution thread

The Linux Signals

- *signals* - the operating system's method of managing software interrupts
- communicate information between the kernel and the running process or between two processes
- several different uses for signals
- most often we are interested in those sent from the kernel due to an unrecoverable error
- once they are issued, they are interpreted and some kind of action is taken

The Linux Signals

- the few most common signals and their meaning are:
  - SIGSEGV - invalid access to memory
  - SIGKILL - process is terminated
  - SIGPIPE - write to a pipe not read by anything
  - SIGINT - interrupt of a process by the user
  - SIGFPE - arithmetic exception
  - SIGABRT, SIGIOT - process aborted (operation cannot be completed)
- for more info, see the man page on *signal*

Determining the Error

- data analysis part of debugging
- identify the commonalities with the failed test cases
- try to narrow down the parts of the code they have in common
- focus attention on this narrow part of the code
- may require algorithm analysis to determine if error is design-based
- important: you must also understand the problem you meant to model

How to Fix??

- problem due to a typo or flaw in the logic?
  - outline the fix and do a paper test to make sure it works
- problem due to an algorithm flaw?
  - analyze the algorithm and determine what needs to be done to repair the problem
- problem occurred due to a poor understanding of the problem?
  - go back to your problem and try to determine where your understanding is lacking

Fixing and Testing the Code

- make repairs only when you are certain they are necessary
- before making them, double-check your understanding
- once certain of the change to be made, back up the current version first
- make the change, then test to make sure the previously failing tests now work
- test for any new errors resulting from the changes made

Debugging Tools

- Several tools available - print statements, code analyzers, debuggers
- brute force method - print statements, suitable for small programs
- larger programs - code analyzers and debuggers are invaluable
- several types of debuggers, each with its own strengths/weaknesses
- at ACEnet - gdb/ddd/valgrind/Lint, PGI's debuggers, Sun tools, TotalView

Debugging Tools - Terminology

- some commonly used terms in debugging:
  - *breakpoint*: suspend execution at a defined point
  - *step*: statement-by-statement code execution
  - *watch*: observe variable or expression during program execution
  - *trace*: output showing the states of the program as it executes

More on Breakpoints

- different types of breakpoints:
  - conditional breakpoint: program stops only if the defined condition is satisfied
  - unconditional breakpoint: program always stops at defined breakpoint
  - data breakpoint: program execution stops when a particular data value or part of memory changes
- useful to check values of variables and move initial debugging point

More on Stepping

- when stepping through your program, you can choose to step:
  - into: enter a procedure
  - over: bypass the procedure
  - out: exit the procedure at that step and return to the calling procedure
- useful to bypass parts of your code that you know work

Print Statements

- useful for small codes and embedded systems
- create a lot of junk both in output streams and code
- slows things down during runtime
- program often fails before printing is finished
- output in this case can be misleading
- for large codes, it can increase debugging time immensely

Lint/Splint

- a code checker, NOT a compiler
- implemented for use with C, not fortran
- use: *splint <name>.c*
- handy to check your code for errors or non-ANSI things before running through the compiler
- also a command line tool

GDB

- the "GNU debugger"
- works with code compiled using any compiler (needs -g flag)
- useful options: *break, watch, step, continue, backtrace, frame*
- works on the command line so useful for times when X11 forwarding is not wise
- has support for regular expressions (in breakpoint setting, maybe other places)
- handy to examine execution/call stack

Examples

- several code examples, some in C and some in Fortran 77 and 90
- copy from /globalscratch/dshaw on courtenay
- unpack using “tar -vxf IntroDebugging.tar”
- a few examples of bad code...

Further Reading

- web pages for the packages listed on a previous slide
- book and webpage by Andreas Zeller called “Why Programs Fail”
- book by John Fusco called: “The Linux Programmers Toolbox”
- ACEnet/your local CRC for further assistance